Phase 2 - Week 6 - Lesson Day 3

Evaluation Metrics

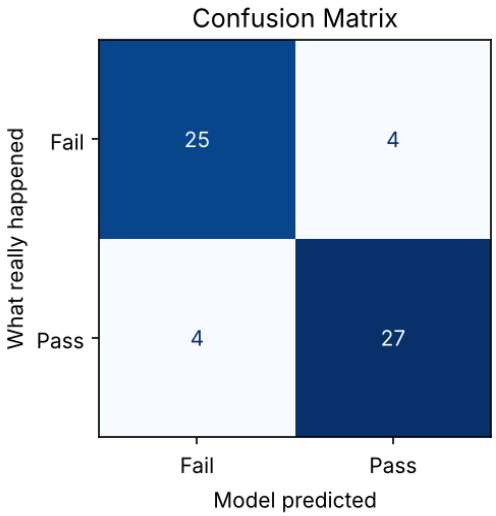
Today's Goal

So far we used one number, **.score()**, to judge a model. That number is accuracy, and it can hide problems. Today you learn the other standard ways to measure a classification model, and when each one matters.

- Read a confusion matrix
- Understand accuracy, precision, recall, and F1
- Understand ROC-AUC
- Know why accuracy alone can fool you

1. The Confusion Matrix

Simple definition: A confusion matrix is a small table that counts how many predictions were right and wrong, split by type. Every other metric today is calculated from these four numbers.



Name	What it means	Example (Pass = positive)
True Positive (TP)	Model said Pass, and it really was Pass	Correct
True Negative (TN)	Model said Fail, and it really was Fail	Correct
False Positive (FP)	Model said Pass, but it really was Fail	Wrong: false alarm
False Negative (FN)	Model said Fail, but it really was Pass	Wrong: missed one

These are the same ideas as Type I (false positive) and Type II (false negative) errors from Week 4.

2. The Four Main Metrics

Metric	Plain English question it answers	Formula
Accuracy	Out of ALL predictions, how many were right?	(TP + TN) / total
Precision	When the model says Pass, how often is it right?	TP / (TP + FP)
Recall	Out of the students who REALLY passed, how many did the model find?	TP / (TP + FN)
F1 score	One number that balances precision and recall	2 x (P x R) / (P + R)

Precision vs recall in real life: imagine a spam filter. High **precision** means that when it says "spam", it is almost always really spam (good emails are safe). High **recall** means it catches almost all the real spam (few slip through). Often you must choose which matters more.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

predictions = model.predict(X_test)

print(accuracy_score(y_test, predictions))
print(precision_score(y_test, predictions))
print(recall_score(y_test, predictions))
print(f1_score(y_test, predictions))
```

3. ROC-AUC

Simple definition: ROC-AUC is one number between 0.5 and 1 that tells you how well the model separates the two groups (Pass and Fail). **0.5** means no better than guessing. **1.0** means perfect separation. Above 0.8 is usually considered good.

It uses the model's **probabilities**, not just its final yes/no answers, so you pass it **predict_proba**, not predict.

```
from sklearn.metrics import roc_auc_score

probabilities = model.predict_proba(X_test)[:, 1]   # probability of Pass
print(roc_auc_score(y_test, probabilities))
```

4. The Accuracy Trap

Imagine 100 bank transactions where only 5 are fraud. A lazy model that always says "not fraud" is right 95 times, so its accuracy is **95%**. It sounds great, but it caught **zero** fraud, so its recall is 0. Accuracy alone fooled us. When one group is much rarer than the other, always check precision and recall too.

Today's Practice

Open **day-3-evaluation-metrics.ipynb**. All answers are already solved.

Practice Questions (5 total - all solved)

1. Confusion matrix

Train a Random Forest and print its confusion matrix on the test data.

2. The four metrics

Print accuracy, precision, recall, and F1 using Scikit-learn.

3. By hand

Calculate precision and recall yourself from the confusion matrix and check they match Scikit-learn.

4. ROC-AUC

Calculate the ROC-AUC score using predict_proba.

5. The accuracy trap

Build 100 fake transactions with only 5 frauds. Show that a model that always says 'not fraud' gets 95% accuracy but 0 recall.

Once you and Halimo finish, move to Lesson Day 4 (Model Comparison and Mini Challenge).